Assignment 3 Graphs

Yerkebulan Niyalov | SE2526 | Design and Analysis of Algorithms

The project implements the required graph algorithms in Java. The full benchmark matches all ten rows of the assignment reference table. Kruskal and Prim agree on every generated graph. The main evidence below is operation counts; execution time is only supporting information.

1. Implementation and complexity

One adjacency-list Graph class stores directed and undirected graphs. Each logical edge is also kept once in an edge list. DSU uses union by size and iterative path compression. Prim uses a PriorityQueue of edges. Kosaraju uses an explicit stack. Disconnected MST inputs return a forest, and DAG algorithms reject cycles. Weights are int, but totals and distances are long.

Algorithm	Time	Extra space	Reason
DSU find / union	O(alpha(V)) amortized	O(V) DSU storage; O(1) per call	Size-based union and path compression flatten trees. Initialization takes O(V).
Kruskal	O(V + E log E)	O(V + E)	Copy and sort edges; DSU checks cost O(E alpha(V)).
Prim (lazy heap)	O(V + E log(E + 1))	O(V + E)	Every adjacency entry is scanned; up to O(E) queued edges are inserted and removed.
Kosaraju	O(V + E)	O(V + E)	Two DFS passes and a reversed graph; arrays and stack use O(V).
Kahn	O(V + E)	O(V)	Compute in-degrees, enqueue each vertex once and process each outgoing edge once.
DAG shortest paths	O(V + E)	O(V)	Topological sorting plus one relaxation pass; distance and parent arrays.

For a simple graph, E is at most V squared, so Prim is also usually written as O((V + E) log V). This implementation allows parallel edges and uses a lazy edge heap, so log E is the more precise bound. alpha(V) is the inverse Ackermann function: it grows so slowly that DSU operations are almost constant in practice, but the bound is amortized over a sequence.

Counter definitions

dsu_ops counts find calls and successful unions. comparisons counts weight comparisons made by the sort or heap. edge_scans counts the main traversal: Kruskal's considered edges, Prim's adjacency entries, the two SCC DFS passes, Kahn's queue processing and the DAG relaxation pass. max_depth records the largest explicit DFS stack size and is not added to operation totals.

To match the assignment's exact scan counts, reverse-graph construction, initial in-degree calculation, topological preprocessing for paths and validation are excluded from edge_scans. They are still real O(V + E) work and are included in runtime where performed inside the algorithm. Thus a scan count is a defined part of the work, not every instruction executed.

2. Correctness arguments

Cut property and the two MST algorithms

Consider a cut that does not split any component of the forest already selected. Let e be a lightest edge crossing that cut. If a minimum spanning tree does not contain e, adding e creates a cycle. The cycle contains another edge f crossing the same cut, and the weight of f is at least the weight of e. Replacing f with e preserves connectivity and cannot increase the total weight, so some minimum spanning tree contains e. Kruskal uses this property when it selects the next lightest edge joining two DSU components. Prim uses it when it selects the lightest edge leaving its current tree; restarting on each unvisited component gives a minimum spanning forest.

Induction on topological order

In a topological order, every predecessor of a vertex appears before that vertex. Initialize every zero-in-degree vertex to distance zero, which is equivalent to adding a virtual source with zero-weight edges to all such vertices. For the induction hypothesis, assume all vertices before v already have their final shortest distances. Every path to v ends with an edge from one of these earlier vertices, so all candidates for its distance have been relaxed before v is processed. Therefore the minimum candidate stored at v is its final shortest distance, and relaxing its outgoing edges preserves the induction. Negative edges do not invalidate this reasoning because a DAG has no cycle that can send a better path back to an already processed vertex. A general graph needs another method: negative edges can break Dijkstra's greedy choice, and a reachable negative cycle can make a finite shortest distance undefined; Bellman-Ford handles negative edges when no such cycle affects the destination.

Tests

Test group	Validation performed
DSU	Random unions against simple set labels; every pair's connectivity and component count; counters and reinitialization.
MST	100 random connected graphs, including negative weights, loops and parallel edges. Equal weights, n - 1 edges, no cycle, valid selected edges. Disconnected input returns three components.
SCC	40 random graphs with up to 200 vertices compared with BFS reachability in both directions. A chain of 100,000 vertices completes with max_depth = 100,000.
Topological order	Every edge respects positions in the order. Cyclic input and a self-loop are rejected; empty and isolated graphs work.
DAG paths	100 random DAGs with negative weights compared with Bellman-Ford. Restored paths, multiple sources, an isolated vertex and long distance sums are checked.
Generator	Both densities at n = 1,000 reproduce the MST, SCC and DAG reference values. The full benchmark checks every larger size too.

The Maven test suite contains 12 test methods and passed with no failures. All three small input files are used by tests. The full benchmark additionally checks equal MST weights, tree sizes, valid topological order and reference results before accepting the CSV.

3. Benchmark results and normalized counters

Each graph starts with a fresh Random(42). The generator shuffles using Fisher-Yates, builds the required undirected spanning tree, and draws weights from 1 to 1,000. When endpoints match, only b is redrawn. Five sizes and two densities give ten cases; three graph kinds are generated per case. Five algorithm results per case produce 50 CSV rows. All reference values below matched exactly.

n	m	MST weight	SCC count	DAG distance sum
1,000	4,000	147,970	27	396,274
1,000	20,000	30,363	1	252,411
3,162	12,648	468,911	137	1,271,911
3,162	63,240	97,589	1	723,157
10,000	40,000	1,486,139	373	3,941,828
10,000	200,000	309,550	1	2,381,342
31,623	126,492	4,715,515	1,188	12,551,891
31,623	632,460	968,517	1	7,376,612
100,000	400,000	15,012,094	3,923	40,096,539
100,000	2,000,000	3,056,681	1	23,512,087

The table below shows the minimum and maximum ratio over the five sizes. E log E uses log base 2. A bounded, roughly stable ratio supports the predicted growth; a finite benchmark is not a proof of an asymptotic bound.

Algorithm / ratio	Sparse min - max	Dense min - max
DSU in Kruskal: dsu_ops / E	2.290 - 2.748	0.565 - 0.894
Kruskal: comparisons / (E log E)	0.745 - 0.907	0.666 - 0.903
Prim: comparisons / (E log E)	1.831 - 1.946	1.826 - 1.892
Kosaraju: scans / (V + E)	1.600 - 1.600	1.905 - 1.905
Kahn: scans / (V + E)	0.800 - 0.800	0.952 - 0.952
DAG paths: scans / (V + E)	0.800 - 0.800	0.952 - 0.952

Traversal ratios are constant because E/V is fixed within each density. Sorting and heap ratios vary with input structure and repeated weights but stay within a small range. DSU call counts measure how often DSU is used; they do not count parent-pointer steps and cannot by themselves verify the alpha(V) bound. Kruskal's early stopping also changes the fraction of edges that need DSU checks.

4. Operation plots and slope check

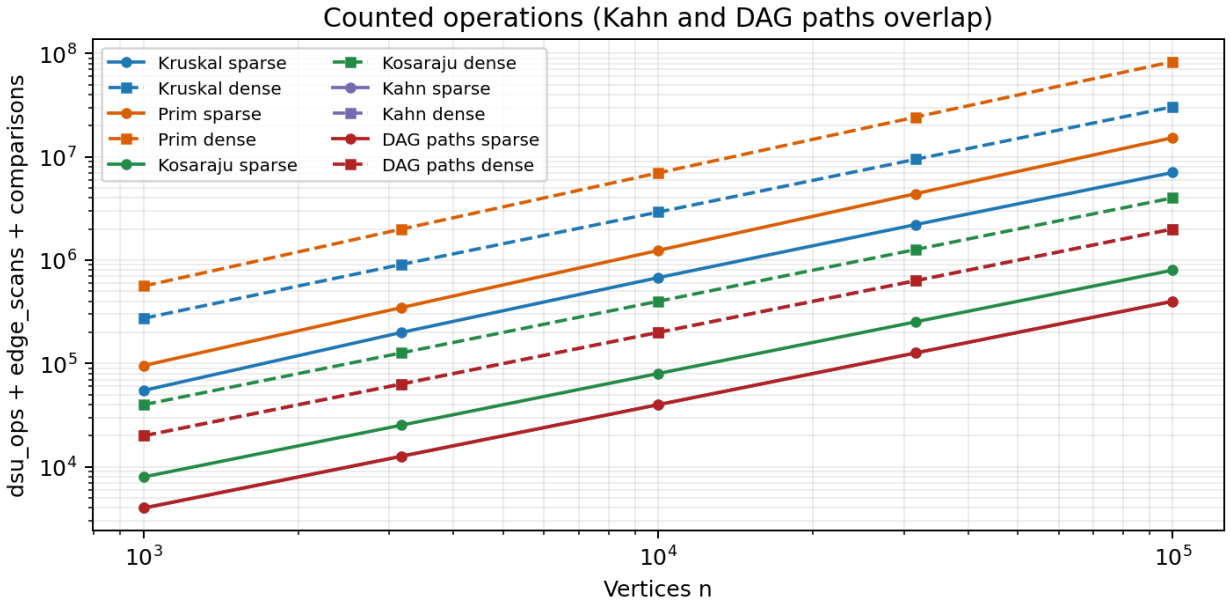


Figure 1. Total counted operations against n, on log-log axes. Kahn and DAG paths overlap because both count m scans. Sparse means m = 4n; dense means m = 20n.

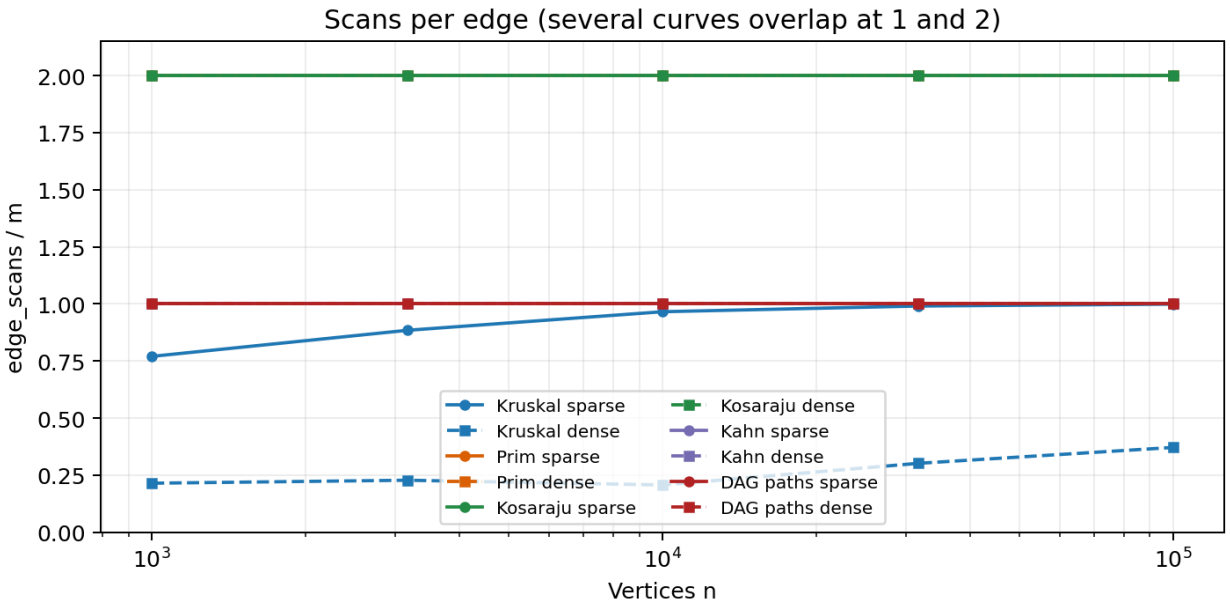


Figure 2. Prim and Kosaraju have scans/m = 2; Kahn and DAG relaxation have scans/m = 1. Equal curves overlap. Kruskal can stop before reaching the end of the sorted edge list.

Fitted log-log slope	Sparse	Dense
Kruskal edge scans	1.055	1.119
Kruskal weight comparisons	1.055	1.016
Prim / Kosaraju / Kahn / DAG scans	1.000	1.000

Slopes are least-squares fits of log(counter) against log(n). All four complete traversals have slope 1.000. Kruskal's log factor belongs to sorting comparisons, not edge scans; scans alone cannot demonstrate E log E. Repeated weights and early termination affect the observed slopes.

5. Algorithm choice and time

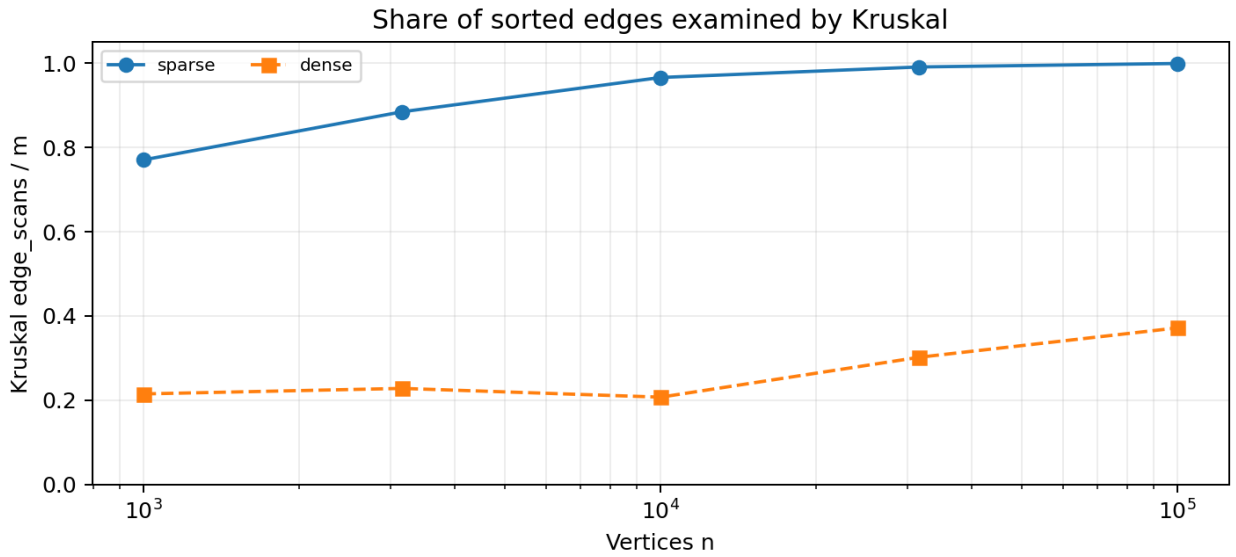


Figure 3. Kruskal examines a smaller share of the dense edge list, but it still sorts every edge. Prim avoids the global sort but scans all adjacency entries and pays for heap operations. Kruskal uses fewer total counted operations than this lazy Prim on all ten inputs. These experiments do not establish an input range where Prim wins.

Situation	Choice and reason
Sparse undirected graph in an edge list	Kruskal: sorting the existing edge list and checking DSU is straightforward.
Undirected graph already in adjacency lists	Prim: grow directly from adjacency lists. For this lazy heap version, compare measured work with Kruskal before claiming an advantage.
Very dense undirected graph	A matrix-based O(V squared) Prim is a common choice, but it is not implemented here. Both required implementations use sorting or a heap.
Directed graph where cycles are possible	Kosaraju for mutually reachable groups; Kahn if the task is to obtain a schedule or detect a cycle.
DAG with weighted dependencies	Topological relaxation: O(V + E), with negative weights allowed and parent-based path restoration.
General graph shortest paths	Use Dijkstra for nonnegative edges or Bellman-Ford when negative edges are possible. DAG relaxation requires acyclicity.

Each CSV time is the median of five runs using System.nanoTime on Java 21.0.7. At n = 1,000 the medians ranged from 0.287 to 8.615 ms. On the largest dense inputs: Kruskal 517.7 ms; Prim 2762.6 ms; Kosaraju 755.0 ms; Kahn 89.7 ms; DAG paths 162.4 ms. Graph generation and CSV writing are outside the timer; algorithm allocations and preprocessing are inside it. There is no separate JVM warm-up stage, so JIT compilation, garbage collection, hardware and background load can affect these numbers, especially on the smallest cases. This is why the conclusions use counters rather than a single timing comparison.

Reproduction: run mvn clean package, then java -Xmx3g -jar target/graphs-1.0.jar, then python scripts/report.py with the Python dependencies installed. The seed reproduces graph results and counters for the same implementation; wall-clock times are expected to change. Source of task requirements and reference values: DAA Assignment 3 Graphs, Taubakabyl Nurlybek. AI assistance is acknowledged in README.md.